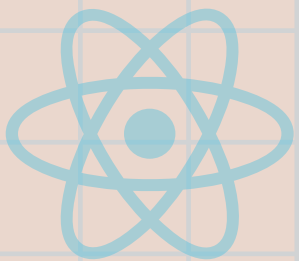


State Management in React

Context API ▪ Redux ▪ LocalStorage
When & Why to Use Each



What is Context API?

Context API is a built-in React feature that allows you to share data globally across components without prop drilling.

Key Points:

- Introduced in React 16.3
- Best for small to medium global states
- Example: Auth state, theme, user preferences

React tree with a single “context provider” wrapping multiple components.

What is Redux?

Redux is a predictable state management library for JavaScript apps, offering a single source of truth (store) and strict unidirectional data flow.

Key Points:

- External library (not built-in)
- Uses actions, reducers, and store
- Great for complex, scalable apps with frequent state updates

Flow: Action → Reducer → Store → UI

When to Use Context API

Use Context API when:

- App is small to medium in scale
- Few global states like theme, auth, language
- You want minimal setup and dependencies
- Avoid for heavy or frequently updated data (may cause re-renders)

Example:

Sharing dark/light mode or logged-in user info.

When to Use Redux

Use Redux when:

- Application has complex or nested state
- You need predictability, debugging, and scalability
- Multiple components depend on the same dynamic data
- Slightly higher learning curve and setup

Example:

Large dashboards, e-commerce carts, admin panels.

Role of LocalStorage

What it does:

- LocalStorage stores data persistently in the browser, even after reload.
- It is not a state manager but a persistence layer.

Use Case:

Save tokens, user preferences, filters

Restore state when the user revisits

Example:

Store JWT token → use Context/Redux for in-memory access.

Why Use Context API or Redux if We Have LocalStorage?

Answer:

- Because LocalStorage is static, while Redux and Context are reactive.
- LocalStorage doesn't trigger UI re-renders
- Redux/Context manage live data flow inside your app
- LocalStorage just keeps a copy of data

In short:

Context/Redux = In-memory state management

LocalStorage = Persistence

Can We Use Only LocalStorage?

Yes, but with limitations.

You can manage basic app data like:

- User preferences
- Recently viewed items
- Simple form persistence

But it fails when you need:

- Frequent updates
- Derived state (computed values)
- Reactive UI updates

Practical Limit:

Good for small apps or POCs — not for complex, multi-page applications.

How They Work Together

Modern apps use all three together:

- Redux/Context API → Manage live, in-memory state
- LocalStorage → Persist that state for reloads

Example Flow:

On app load → Read from LocalStorage → Hydrate Context/Redux store → Sync changes back.

Example: Combined Usage

Auth Example:

- **LocalStorage** → stores user token
- **Context API** → shares user data across app
- **Redux** → manages API calls, loading states, and data

Result: Fast, persistent, and reactive user experience.

Context API → Great for small/medium global states

Redux → Ideal for large, complex, data-heavy apps

LocalStorage → Keeps data persistent across sessions

Combine them wisely — persistence from LocalStorage, reactivity from Context/Redux.

Feature	Context API	Redux	LocalStorage
Type	Built-in React API	External Library	Browser Storage
Purpose	Global state sharing	Complex state management	Persistent storage
Reactivity	✔ Reactive	✔ Highly reactive	✗ Non-reactive
Data Persistence	✗ No	✗ No (needs middleware)	✔ Yes
Scalability	Medium	High	Low
Complexity	Low	High	Low
Debugging Tools	✗ Minimal	✔ Excellent (DevTools)	✗ None
Common Use Cases	Auth, Theme	Dashboards, API-heavy apps	Tokens, Preferences
Setup	Simple	Moderate	Very Simple